Javascript Interview Preparation questions

19 min read Updated 13 Mar 2026 Topic 4 of 23

JavaScript Core Interview Questions

1. What is a Closure?



Even after outerFunction returns, the inner function keeps a live reference to count — that private, surviving variable is the closure.

A **closure** is a feature in JavaScript where an inner function has access to variables from its **outer (lexical) scope**, even after the outer function has finished executing.

In simple terms:

A closure allows a function to **remember and access variables from the scope where it was created**.

Example

```javascript
function outer(x, y) {

  function inner() {
    console.log(x, y);
  }

  return inner;
}

let fn = outer(2, 3);
fn();
```

Output

JAVASCRIPT Copy

```
2 3
```

Explanation

1. `outer()` executes and returns `inner`.
2. Normally `x` and `y` would disappear after `outer()` finishes.
3. But because of **closure**, `inner()` still has access to `x` and `y`.

Common Uses of Closures

- Data privacy
- Function factories
- Memoization
- Event handlers
- Module patterns

2. What are bind(), call(), and apply()?

These methods are used to **control the value of** `this` **inside a function**.

bind()

`bind()` creates a **new function** with `this` permanently bound.

JAVASCRIPT Copy

```
const greet = {
  fname: "Geeky",
  lname: "Frontend",
  sayHello() {
    console.log(this.fname, this.lname);
  }
};

let fn = greet.sayHello.bind(greet);
fn();
```

Output

JAVASCRIPT Copy

```
Geeky Frontend
```

call()

`call()` **immediately executes** the function with a specified `this`.

JAVASCRIPT Copy

```
function sayHello(age) {
  console.log(this.fname, this.lname, age);
```

```
sayHello.call(greet, 25);
```

Output

JAVASCRIPT Copy

```
Geeky Frontend 25
```

apply()

`apply()` works like `call()` but accepts arguments as an **array**.

JAVASCRIPT Copy

```
sayHello.apply(greet, [25]);
```

Output

JAVASCRIPT Copy

```
Geeky Frontend 25
```

3. Difference Between call, apply, and bind

Method	Executes Immediately	Argument Style	Returns Function
call	Yes	Comma separated	No

bind	No	Comma separated	Yes

Example:

```javascript
fn.call(obj, a, b)
fn.apply(obj, [a, b])
fn.bind(obj)(a, b)
```

---

4. What is Prototypal Inheritance?

Prototypal inheritance allows objects to **inherit properties and methods from another object**.

In JavaScript, every object has an internal property called:

```javascript
[[Prototype]]
```

This forms the **prototype chain**.

Example

```javascript
function Animal(name) {
  this.name = name;
}
```

```
};

const dog = new Animal("Tommy");

dog.sayName();
```

Output

JAVASCRIPT Copy

```
Tommy
```

Here:

JAVASCRIPT Copy

```
dog → Animal.prototype → Object.prototype
```

---

5. Different Ways to Create Objects in JavaScript

1. Object Literal

JAVASCRIPT Copy

```
const person = {
  name: "Geeky",
  age: 20
};
```

---

2. Constructor Function

JAVASCRIPT Copy

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

const p = new Person("Geeky", 20);
```

3. ES6 Classes

JAVASCRIPT Copy

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
}

const p = new Person("Geeky", 20);
```

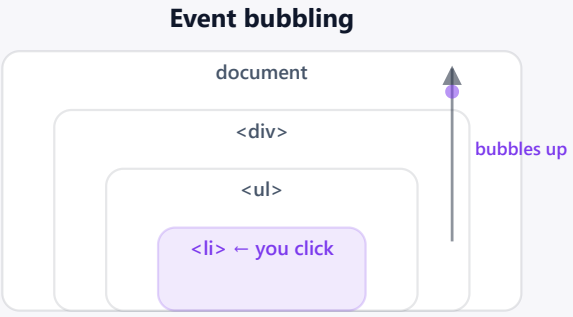
4. Object.create()

JAVASCRIPT Copy

```
const person = {
  name: "Geeky"
};

const newPerson = Object.create(person);
```

6. Event Bubbling and Event Delegation



A click on the innermost element fires there first, then bubbles outward through every ancestor. Event delegation puts one listener on a parent to catch them all.

Event Bubbling

Event bubbling means the event travels **from child → parent elements**.

Example DOM:

JAVASCRIPT Copy

```
button → inner div → outer div
```

Example:

JAVASCRIPT Copy

```
button.addEventListener("click", () => {
  console.log("Button clicked");
});

inner.addEventListener("click", () => {
  console.log("Inner clicked");
});

outer.addEventListener("click", () => {
  console.log("Outer clicked");
});
```

Output:

JAVASCRIPT Copy

```
Button clicked
Inner clicked
Outer clicked
```

Event Delegation

Event delegation attaches **one event listener to a parent element**.

Example:

JAVASCRIPT Copy

```
const list = document.querySelector("#list");

list.addEventListener("click", (event) => {
  if (event.target.tagName === "LI") {
    console.log(event.target.textContent);
  }
});
```

Benefits:

- Better performance
- Handles dynamic elements
- Less memory usage

---

7. What is Hoisting?

Hoisting means variable and function declarations are moved to the top of their scope during execution context creation.

Example:

JAVASCRIPT Copy

```
console.log(x);
var x = 5;
```

Output

JAVASCRIPT Copy

```
undefined
```

Actual behavior:

JAVASCRIPT Copy

```
var x;
console.log(x);
x = 5;
```

---

let and const Hoisting

They are hoisted but placed in the **Temporal Dead Zone (TDZ)**.

JAVASCRIPT Copy

```
console.log(a);
let a = 5;
```

Error:

```
ReferenceError
```

---

8. What is the `this` keyword?

`this` refers to the **object that is executing the current function**.

Global Context

JAVASCRIPT Copy

```javascript
console.log(this);
```

Browser → `window`

Object Method

JAVASCRIPT Copy

```javascript
const person = {
  name: "John",
  greet() {
    console.log(this.name);
  }
};

person.greet();
```

Output

JAVASCRIPT Copy

Constructor

JAVASCRIPT Copy

```javascript
function Person(name) {
  this.name = name;
}

const p = new Person("John");
```

`this` refers to the **new instance**.

9. Difference Between Classes and Functions

Feature	Function Constructor	Class
Syntax	Older	ES6
Hoisting	Yes	No
Strict Mode	Optional	Always
Inheritance	Prototype	extends

Example:

JAVASCRIPT Copy

```
class User {
  constructor(name) {
    this.name = name;
  }
}
```

10. Difference Between prototype and proto

prototype

- Property of **constructor functions**

Example:

JAVASCRIPT Copy

```
function Animal() {}

Animal.prototype.sayHi = function () {};
```

proto

- Property of **every object**
- Points to its prototype

Example:

JAVASCRIPT Copy

```
const obj = {};

obj.__proto__ === Object.prototype
```

Relationship

JAVASCRIPT Copy

```
instance.__proto__ === Constructor.prototype
```

Example

JAVASCRIPT Copy

```
function Animal(){}

const dog = new Animal();

console.log(dog.__proto__ === Animal.prototype);
```

Output

JAVASCRIPT Copy

```
true
```

JavaScript Core Interview Questions (11–20)

11. What is Symbol in JavaScript?

A **Symbol** is a primitive data type introduced in **ES6**.

Symbols are used to create **unique identifiers**.

Even if two symbols have the same description, they are **not equal**.

Example

JAVASCRIPT Copy

```
const sym1 = Symbol("id");
const sym2 = Symbol("id");

console.log(sym1 === sym2);
```

Output

JAVASCRIPT Copy

```
false
```

Each symbol is **completely unique**.

Use Case

Symbols are often used as **object keys** to avoid property name collisions.

JAVASCRIPT Copy

```
const id = Symbol("id");

const user = {
  name: "Geeky Frontend",
  [id]: 101
};
```

Output

JAVASCRIPT Copy

```
101
```

12. What is an Iterator?

An **iterator** is an object that allows you to **iterate over a collection one value at a time**.

JavaScript collections like arrays implement the **iterator protocol**.

An iterator must have a method:

JAVASCRIPT Copy

```
next()
```

Which returns:

JAVASCRIPT Copy

```
{ value: any, done: boolean }
```

Example

JAVASCRIPT Copy

```
const arr = [1, 2, 3];
```

```
console.log(iterator.next());
console.log(iterator.next());
console.log(iterator.next());
```

Output

JAVASCRIPT Copy

```
{ value: 1, done: false }
{ value: 2, done: false }
{ value: 3, done: false }
{ value: undefined, done: true }
```

---

13. What is a Generator?

A **generator** is a special function that can **pause and resume execution**.

Generators use:

JAVASCRIPT Copy

```
function*
```

and the keyword

JAVASCRIPT Copy

```
yield
```

Example

```javascript
function* counter() {
  let i = 0;

  while (true) {
    yield i;
    i++;
  }
}

const gen = counter();

console.log(gen.next().value);
console.log(gen.next().value);
console.log(gen.next().value);
```

Output

JAVASCRIPT Copy

```
0
1
2
```

Common Uses

- Lazy evaluation
- Iterating large datasets
- Custom iterators
- Async workflows

Example: Chunk Array Using Generator

```javascript
function* chunkArray(array, size) {

  for (let i = 0; i < array.length; i += size) {
    yield array.slice(i, i + size);
  }

}

const data = [1,2,3,4,5,6,7,8];

for (const chunk of chunkArray(data, 3)) {
  console.log(chunk);
}
```

Output

JAVASCRIPT Copy

```
[1,2,3]
[4,5,6]
[7,8]
```

14. What is the `new` Keyword?

The `new` keyword is used to create **instances of objects from constructor functions or classes**.

When `new` is used, JavaScript performs the following steps:

1. Creates an empty object `{}`
2. Sets `this` to the new object
3. Links the object to the constructor's prototype
4. Executes the constructor

Example

JAVASCRIPT Copy

```
function User(name) {
  this.name = name;
}

const user = new User("Geeky Frontend");

console.log(user.name);
```

Output

JAVASCRIPT Copy

```
Geeky Frontend
```

15. What are async and await?

`async/await` is syntax built on top of **Promises** that makes asynchronous code **look synchronous**.

Example

JAVASCRIPT Copy

```
async function fetchData() {

  const response = await fetch("https://api.example.com/data");
```

```
  return data;
}
```

Here:

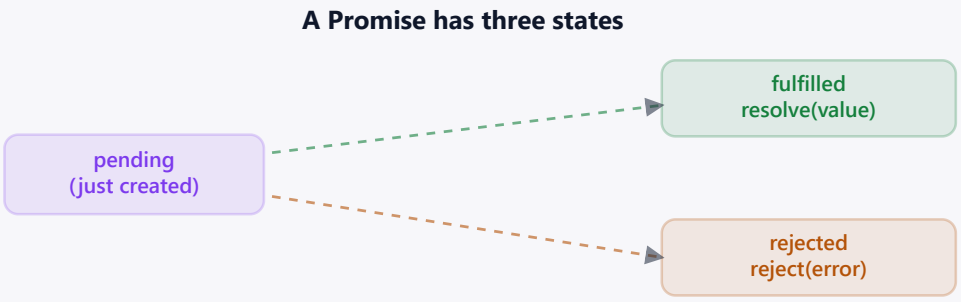
JAVASCRIPT Copy

```
await pauses execution until promise resolves
```

Benefits

- cleaner syntax
- easier error handling
- avoids deep `.then()` chains

16. What is a Promise?



A promise starts pending and settles exactly once — to fulfilled or rejected. After that it never changes again.

A Promise represents the eventual result of an asynchronous operation.

A promise has three states:

JAVASCRIPT Copy

```
pending
fulfilled
rejected
```

Example

JAVASCRIPT Copy

```
const myPromise = new Promise((resolve, reject) => {

  let condition = true;

  if (condition) {
    resolve("Promise resolved");
  } else {
    reject("Promise rejected");
  }

});

myPromise
  .then(result => console.log(result))
  .catch(error => console.log(error));
```

Important Promise Methods

Promise.all()

Resolves when **all promises succeed**.

```
Promise.all([p1, p2, p3])
```

If one fails → entire promise fails.

Promise.allSettled()

Returns results of **all promises** regardless of success or failure.

JAVASCRIPT Copy

```
Promise.allSettled([p1, p2, p3])
```

---

17. async/await vs Promises

Feature	Promises	Async/Await
Syntax	`.then()` chains	synchronous style
Readability	harder for long chains	easier
Error Handling	`.catch()`	`try/catch`
Debugging	harder	easier

Promise Example

JAVASCRIPT Copy

```
getUserData()
  .then(user => getPosts(user.id))
  .then(posts => console.log(posts))
  .catch(err => console.error(err));
```

Async/Await Example

JAVASCRIPT Copy

```
async function loadData() {

  try {

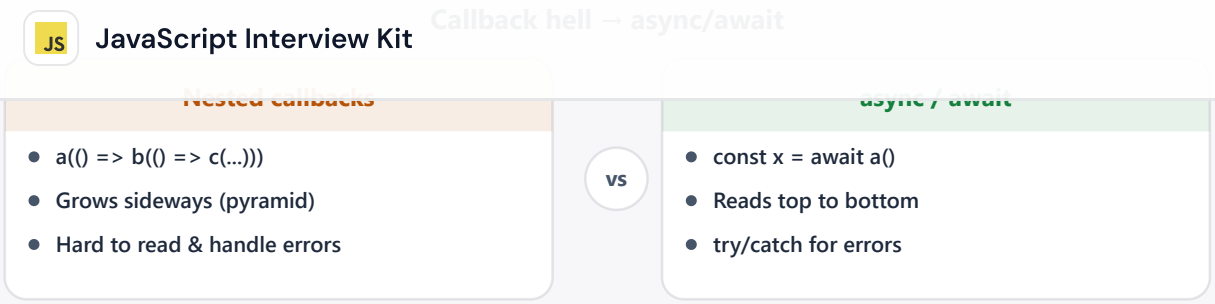
    const user = await getUserData();
    const posts = await getPosts(user.id);

    console.log(posts);

  } catch(err) {
    console.error(err);
  }

}
```

18. What is Callback Hell?



Promises and async/await flatten the "pyramid of doom" into linear, readable code.

Callback hell occurs when multiple asynchronous operations are nested inside each other.

This creates deeply nested code that is difficult to read and maintain.

Example

JAVASCRIPT Copy

```
asyncCall1(function(result1) {
  asyncCall2(result1, function(result2) {
    asyncCall3(result2, function(result3) {
      console.log(result3);
    });
  });
});
```

This structure is often called:

JAVASCRIPT Copy

```
"Pyramid of Doom"
```

Solution

- Promises
- async/await

---

19. What are Higher Order Functions and Callbacks?

JavaScript functions are **first-class citizens**.

Meaning they can:

- be passed as arguments
- be returned from functions
- be stored in variables

Callback Function

A function passed into another function.

JAVASCRIPT Copy

```
function greet(name) {
  console.log("Hello " + name);
}

function processUser(callback) {
  callback("Geeky Frontend");
}

processUser(greet);
```

Higher Order Function

A function that **accepts another function as argument**.

Example:

JAVASCRIPT Copy

```
map()
filter()
reduce()
```

---

20. What is Currying?

Currying is the technique of **transforming a function with multiple arguments into a series of functions with single arguments**.

JAVASCRIPT Copy

```
f(a,b,c)
```

becomes

JAVASCRIPT Copy

```
f(a)(b)(c)
```

Example

JAVASCRIPT Copy

```
  return function(b) {
    return function(c) {
      return a + b + c;
    };
  };
}

console.log(currySum(2)(3)(4));
```

Output

JAVASCRIPT Copy

```
9
```

Practical Use

Currying is useful for:

- function composition
- partial application
- functional programming patterns

JavaScript Core Interview Questions (21–30)

21. What is a Polyfill and a Transpiler (Babel)?

Polyfill

A **polyfill** is code that provides modern JavaScript functionality in **older browsers that do not support it**.

Example: If an older browser does not support `Math.trunc`.

JAVASCRIPT Copy

```
if (!Math.trunc) {
  Math.trunc = function (n) {
    return n < 0 ? Math.ceil(n) : Math.floor(n);
  };
}
```

Now older browsers can use:

JAVASCRIPT Copy

```
Math.trunc(4.7); // 4
```

Transpiler

A **transpiler** converts modern JavaScript code into older JavaScript syntax.

Example tool:

JAVASCRIPT Copy

```
Babel
```

Example modern code:

```
const height = height ?? 100;
```

Babel converts it into:

JAVASCRIPT Copy

```
const height = (height !== undefined && height !== null) ? height : 100;
```

This allows modern code to run in **older browsers**.

---

22. Important Features Introduced in ES6 (ES2015)

ES6 introduced many major improvements to JavaScript.

Arrow Functions

Shorter syntax for writing functions.

JAVASCRIPT Copy

```
const greet = () => console.log("Hello Geeky Frontend");
```

---

let and const

Block-scoped variables.

JAVASCRIPT Copy

```
let count = 10;
```

Template Literals

String interpolation.

JAVASCRIPT Copy

```javascript
const name = "Geeky Frontend";
console.log(`Hello ${name}`);
```

Destructuring

Extract values from arrays or objects.

JAVASCRIPT Copy

```javascript
const user = { name: "Geeky Frontend", age: 25 };

const { name, age } = user;
```

Spread Operator

JAVASCRIPT Copy

```javascript
const arr = [1,2,3];
const copy = [...arr];
```

Promises

Better async handling.

JAVASCRIPT Copy

```
fetch("/api")
  .then(res => res.json())
```

Modules

JAVASCRIPT Copy

```
export function greet(){}
import { greet } from "./file.js";
```

Map and Set

New collection types.

JAVASCRIPT Copy

```
const set = new Set([1,2,3]);
```

23. What is ECMAScript?

ECMAScript is the official specification that defines how JavaScript should work.

Organization:

JAVASCRIPT Copy

JavaScript follows the **ECMAScript standard**.

Example versions:

Version	Year	Key Features
ES5	2009	strict mode
ES6	2015	classes, arrow functions
ES2020	2020	optional chaining

Example ES2020 feature:

BigInt

JAVASCRIPT Copy

```
const big = 12345678901234567890n;
```

24. Difference Between Function Declaration, Expression, Anonymous Function and Arrow Function

JAVASCRIPT Copy

```
function greet(){
  console.log("Hello Geeky Frontend");
}
```

Hoisted.

Function Expression

JAVASCRIPT Copy

```
const greet = function(){
  console.log("Hello Geeky Frontend");
};
```

Not hoisted.

Anonymous Function

A function without a name.

JAVASCRIPT Copy

```
setTimeout(function(){
  console.log("Hello");
},1000);
```

Arrow Function

Short syntax introduced in ES6.

JAVASCRIPT Copy

```
const greet = () => {
  console.log("Hello Geeky Frontend");
};
```

25. How `this` behaves in Arrow Functions

Arrow functions **do not have their own** `this`.

They inherit `this` from the **lexical scope**.

Example

JAVASCRIPT Copy

```
const obj = {

  name: "Geeky Frontend",

  regular() {
    console.log(this.name);
  },

  arrow: () => {
    console.log(this.name);
  }

};

obj.regular();
obj.arrow();
```

Output:

JAVASCRIPT Copy

```
Geeky Frontend
undefined
```

Why?

Arrow function's `this` refers to **global scope**, not the object.

---

26. Custom Implementation of map, filter and reduce

Custom map

JAVASCRIPT Copy

```
Array.prototype.myMap = function(callback){

  const result = [];

  for(let i=0;i<this.length;i++){
    result.push(callback(this[i], i, this));
  }

  return result;

};
```

Custom filter

JAVASCRIPT Copy

```
  const result = [];

  for(let i=0;i<this.length;i++){
    if(callback(this[i], i, this)){
      result.push(this[i]);
    }
  }

  return result;

};
```

Custom reduce

JAVASCRIPT Copy

```
Array.prototype.myReduce = function(callback, initialValue){

  let accumulator = initialValue;

  for(let i=0;i<this.length;i++){

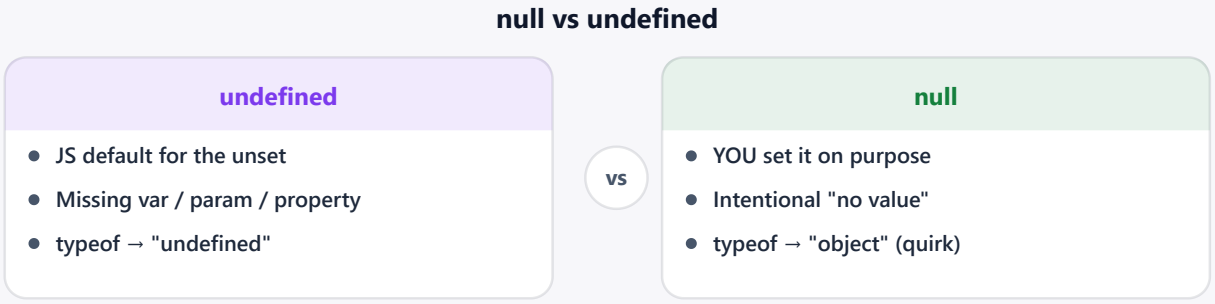
    if(accumulator !== undefined){
      accumulator = callback(accumulator, this[i], i, this);
    } else {
      accumulator = this[i];
    }

  }

  return accumulator;

};
```

27. Difference Between null and undefined



undefined = “not assigned yet”; null = “explicitly empty”.

Feature	null	undefined
Meaning	intentional absence	value not assigned
Type	object	undefined
Assigned by	developer	JavaScript engine

Example:

JAVASCRIPT Copy

```
let a;
console.log(a); // undefined

let b = null;
console.log(b); // null
```

28. Why Minified Code is Used in Production?

Minification removes:

- whitespace
- comments
- unnecessary characters

Example:

Before

JAVASCRIPT Copy

```
function add(a, b) {
  return a + b;
}
```

After

JAVASCRIPT Copy

```
function add(a,b){return a+b}
```

Benefits:

- smaller bundle size
- faster loading
- lower bandwidth usage

Often reduces file size by **30–60%**.

29. Difference Between prototype and proto

- Property of **constructor functions**

Example:

JAVASCRIPT Copy

```
function User(){}

User.prototype.sayHi = function(){
  console.log("Hello Geeky Frontend");
};
```

proto

- Property of **all JavaScript objects**
- Points to the object's prototype

Example

JAVASCRIPT Copy

```
const user = new User();

console.log(user.__proto__ === User.prototype);
```

Output

JAVASCRIPT Copy

```
true
```

Prototype chain example:

```
object → constructor.prototype → Object.prototype → null
```

30. Scope vs Context in JavaScript

Scope

Scope determines **variable visibility**.

Types of scope:

Global Scope

JAVASCRIPT Copy

```
let name = "Geeky Frontend";
```

Function Scope

JAVASCRIPT Copy

```
function test(){
  var x = 10;
}
```

Block Scope

JAVASCRIPT Copy

```
}
```

Context

Context refers to **the value of** `this`.

Example:

JAVASCRIPT Copy

```
const obj = {
  foo() {
    return this;
  }
};

obj.foo(); // obj
```

Another example:

JAVASCRIPT Copy

```
function test(){
  console.log(this);
}

test(); // window (browser)
```

Context depends on **how the function is called**.

JavaScript Core Interview Questions (31–40)

31. What is Method Chaining?

Method chaining means calling multiple methods on the same object in a single statement.

Each method returns the object itself (`this`) so the next method can be called.

Example

JAVASCRIPT Copy

```
class Arithmetic {

  constructor() {
    this.value = 0;
  }

  add(n) {
    this.value += n;
    return this;
  }

  multiply(n) {
    this.value *= n;
    return this;
  }

  divide(n) {
    this.value /= n;
    return this;
  }

}

const calc = new Arithmetic();

calc.add(10).multiply(2).divide(5);
```

Output

JAVASCRIPT Copy

```
4
```

32. Is JavaScript Compiled or Interpreted?

JavaScript is often described as an **interpreted language**, but modern engines actually **compile it just-in-time (JIT)**.

Example engines:

- V8 (Chrome, Node.js)
- SpiderMonkey (Firefox)
- JavaScriptCore (Safari)

Process:

JAVASCRIPT Copy

```
JavaScript Code
      ↓
Parsing
      ↓
Compilation (JIT)
      ↓
Execution
```

So JavaScript is technically **JIT compiled**.

Both methods are used to **restrict modifications to objects**.

Object.freeze()

Makes the object **completely immutable**.

You cannot:

- add properties
- remove properties
- modify values

Example

JAVASCRIPT Copy

```
const person = {
  name: "Geeky Frontend",
  age: 25
};

Object.freeze(person);

person.name = "New Name";

console.log(person.name);
```

Output

JAVASCRIPT Copy

```
Geeky Frontend
```

Prevents **adding or removing properties**, but allows modifying existing values.

Example

JAVASCRIPT Copy

```
const person = {
  name: "Geeky Frontend",
  age: 25
};

Object.seal(person);

person.name = "Frontend Geek";

console.log(person.name);
```

Output

JAVASCRIPT Copy

```
Frontend Geek
```

Difference Table

Feature	freeze	seal
Add property	✗	✗
Delete property	✗	✗

34. Why Can const Objects Be Mutated?

`const` prevents **reassignment of the variable**, not modification of the object.

Example

JAVASCRIPT Copy

```
const user = {
  name: "Geeky Frontend"
};

user.name = "Frontend Geek";

console.log(user.name);
```

Output

JAVASCRIPT Copy

```
Frontend Geek
```

This works because:

JAVASCRIPT Copy

```
const locks the reference
not the object's contents
```

To prevent mutation use:

JAVASCRIPT Copy

35. Difference Between Object.create() and {}

{}

Creates a new object using `Object.prototype`.

JAVASCRIPT Copy

```
const obj = {};
```

Prototype chain:

JAVASCRIPT Copy

```
obj → Object.prototype → null
```

Object.create()

Creates a new object with a **custom prototype**.

Example

JAVASCRIPT Copy

```
const person = {
  name: "Geeky Frontend"
};

const user = Object.create(person);
```

Output

JAVASCRIPT Copy

```
Geeky Frontend
```

Here the object inherits properties through the prototype chain.

---

36. Difference Between window and document

window

The **global object in browsers**.

Contains:

- global variables
- browser APIs
- timers
- location
- history

Example

JAVASCRIPT Copy

```
console.log(window.location);
```

Represents the **HTML DOM of the webpage**.

Example

JAVASCRIPT Copy

```
document.querySelector("p");
```

Relationship

JAVASCRIPT Copy

```
window
    └── document
```

37. Difference Between innerHTML and innerText

innerHTML

Returns or sets **HTML content including tags**.

Example

JAVASCRIPT Copy

```
element.innerHTML = "<b>Geeky Frontend</b>";
```

Rendered as:

JAVASCRIPT Copy

innerText

Returns **only visible text**.

Example

JAVASCRIPT Copy

```
element.innerText = "<b>Geeky Frontend</b>";
```

Rendered as:

JAVASCRIPT Copy

```
<b>Geeky Frontend</b>
```

Comparison

Feature	innerHTML	innerText
Parses HTML	Yes	No
Returns tags	Yes	No
Performance	slower	faster

38. Difference Between HTMLCollection and NodeList

Both represent collections of DOM elements.

NodeList

Returned by:

JAVASCRIPT Copy

```
querySelectorAll()
```

Supports:

JAVASCRIPT Copy

```
forEach()
```

Example

JAVASCRIPT Copy

```
const nodes = document.querySelectorAll("div");

nodes.forEach(node => console.log(node));
```

HTMLCollection

Returned by:

```
getElementsByClassName()
getElementsByTagName()
```

Does **not support forEach directly**.

Example

JAVASCRIPT Copy

```javascript
const elements = document.getElementsByClassName("box");

Array.from(elements).forEach(el => console.log(el));
```

Key Difference

Feature	NodeList	HTMLCollection
Live collection	Sometimes	Always
forEach support	Yes	No

39. What is Tree Shaking?

Tree shaking is a technique used by bundlers to **remove unused code from the final bundle**.

Example bundlers:

- Webpack
- Rollup
- Vite

Example

JAVASCRIPT Copy

```
export function add(){}

export function multiply(){}
```

If only `add()` is used:

JAVASCRIPT Copy

```
multiply() is removed during bundling
```

Benefits:

- smaller bundle size
- faster load time
- better performance

---

40. Difference Between Set and WeakSet

Set

Stores **unique values**.

Supports primitives and objects.

JAVASCRIPT Copy

```
const set = new Set();

set.add(1);
set.add(2);
set.add(2);

console.log(set);
```

Output

JAVASCRIPT Copy

```
{1,2}
```

WeakSet

Stores **only objects**.

Objects are held **weakly**, meaning they can be garbage collected.

Example

JAVASCRIPT Copy

```
const weakSet = new WeakSet();

let obj = {};

weakSet.add(obj);

console.log(weakSet.has(obj));
```

Output

```
true
```

If `obj` is removed:

JAVASCRIPT Copy

```
garbage collector removes it from WeakSet
```

Difference Table

Feature	Set	WeakSet
Store primitives	Yes	No
Store objects	Yes	Yes
Iterable	Yes	No
Garbage collected automatically	No	Yes

JavaScript Core Interview Questions (41–50)

41. Service Worker vs Web Worker

Both allow running code outside the main JavaScript thread, but they serve different purposes.

Service Worker

A **Service Worker** is a background script that acts as a **proxy between the browser and the network**.

It can:

- cache resources
- enable offline applications
- intercept network requests
- handle push notifications

Example

Registering a service worker:

JAVASCRIPT Copy

```
navigator.serviceWorker.register("serviceWorker.js");
```

Service worker file:

JAVASCRIPT Copy

```
self.addEventListener("fetch", event => {
  event.respondWith(
    caches.match(event.request)
  );
});
```

Example use case:

JAVASCRIPT Copy

```
Offline web applications
```

Web Worker

A **Web Worker** runs JavaScript in a **separate background thread**.

It is used for **CPU-intensive tasks** without blocking the UI.

Example tasks:

- image processing
- large calculations
- data processing

Example

Main thread

JAVASCRIPT Copy

```
const worker = new Worker("worker.js");

worker.postMessage("start work");

worker.onmessage = function(e) {
  console.log(e.data);
};
```

Worker file (`worker.js`)

```
onmessage = function() {

  let sum = 0;

  for(let i=0;i<1e9;i++){
    sum += i;
  }

  postMessage(sum);

};
```

Difference Table

Feature	Service Worker	Web Worker
Runs in background	Yes	Yes
Network proxy	Yes	No
Offline support	Yes	No
Heavy computations	No	Yes

42. How to Make Object Properties Immutable?

You can make properties immutable using:

JAVASCRIPT Copy

```
Object.defineProperty()
```

Example

JAVASCRIPT Copy

```
const user = {
  firstname: "Geeky",
  lastname: "Frontend"
};

Object.defineProperty(user, "firstname", {
  writable: false,
  configurable: false
});

user.firstname = "New Name";

console.log(user.firstname);
```

Output

JAVASCRIPT Copy

```
Geeky
```

Property cannot be changed.

---

43. What is Object.is()?

`Object.is()` checks whether **two values are exactly the same**.

It is similar to `===` but handles special cases better.

Example

JAVASCRIPT Copy

```
Object.is(1,1)
```

Output

JAVASCRIPT Copy

```
true
```

Special cases

JAVASCRIPT Copy

```
Object.is(NaN, NaN)
```

Output

JAVASCRIPT Copy

```
true
```

But

JAVASCRIPT Copy

```
NaN === NaN → false
```

Another example

JAVASCRIPT Copy

```
Object.is(-0, +0)
```

Output

JAVASCRIPT Copy

```
false
```

44. Difference Between event.target and event.currentTarget

event.target

Refers to the **actual element that triggered the event**.

event.currentTarget

Refers to the **element on which the event listener is attached**.

Example

HTML Copy

```
<div id="outer">
  <button id="btn">Click</button>
</div>
```

```
document.getElementById("outer")
.addEventListener("click", function(e){

  console.log(e.target);
  console.log(e.currentTarget);

});
```

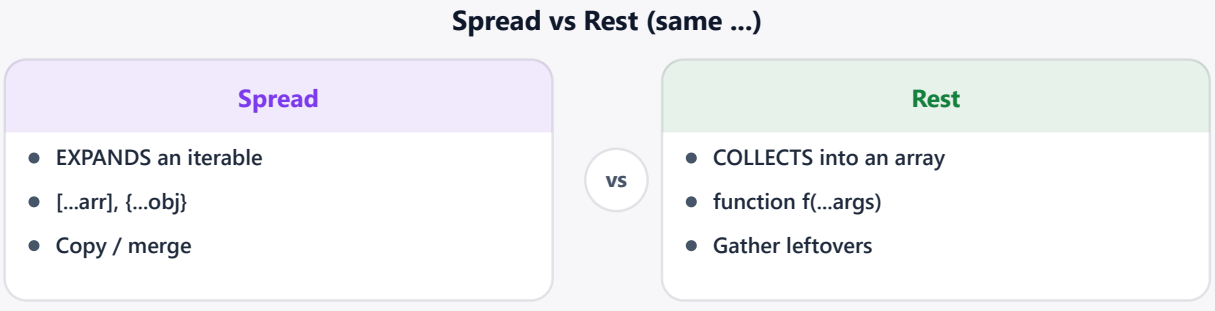
If the button is clicked:

JAVASCRIPT Copy

```
target → button
currentTarget → outer div
```

---

45. Spread vs Rest Operator



Same three dots — spread spreads a collection out, rest gathers arguments in.

Both use `...` but serve different purposes.

Spread Operator

Used to **expand elements**.

JAVASCRIPT Copy

```
const arr = [1,2,3];

const copy = [ ... arr];

console.log(copy);
```

Output

JAVASCRIPT Copy

```
[1,2,3]
```

Rest Operator

Used to **collect multiple arguments into one array**.

Example

JAVASCRIPT Copy

```
function sum( ... nums){
  console.log(nums);
}

sum(1,2,3,4);
```

Output

JAVASCRIPT Copy

```
[1,2,3,4]
```

Difference

Operator	Purpose
Spread	expand values
Rest	collect values

46. How to Deep Freeze an Object?

Deep freeze ensures **nested objects also become immutable**.

Implementation

JAVASCRIPT Copy

```
function deepFreeze(obj){

  const propNames = Object.getOwnPropertyNames(obj);

  for(const name of propNames){

    const value = obj[name];

    if(value && typeof value === "object"){
      deepFreeze(value);
    }

  }
```

Example

JAVASCRIPT Copy

```
const person = {
  name: "Geeky Frontend",
  profession: {
    role: "Developer"
  }
};

deepFreeze(person);

person.profession.role = "Designer";

console.log(person.profession.role);
```

Output

JAVASCRIPT Copy

```
Developer
```

47. What is Pipe and Compose?

Both are **functional programming utilities** used to combine functions.

Pipe

Executes functions **left → right**.

Example

```javascript
const pipe = (...fns) => x =>
  fns.reduce((v,f) => f(v), x);
```

Usage

```javascript
pipe(
  x => x + 2,
  x => x * 3
)(5);
```

Result

```javascript
21
```

Compose

Executes functions **right → left**.

```javascript
const compose = (...fns) => x =>
  fns.reduceRight((v,f) => f(v), x);
```

Difference

pipe	left → right
compose	right → left

48. How to Deep Copy an Object Without Built-in Methods?

A deep copy duplicates **nested objects as well**.

Implementation

JAVASCRIPT Copy

```
function deepCopy(obj){

  const newObj = Array.isArray(obj) ? [] : {};

  for(let key in obj){

    if(typeof obj[key] === "object" && obj[key] !== null){
      newObj[key] = deepCopy(obj[key]);
    }
    else{
      newObj[key] = obj[key];
    }

  }

  return newObj;
```

Example

JAVASCRIPT Copy

```
const car = {
  brand: "BMW",
  year: 2018,
  location: {
    city: "Bangalore"
  }
};

const copy = deepCopy(car);

copy.location.city = "Delhi";

console.log(car.location.city);
```

Output

JAVASCRIPT Copy

```
Bangalore
```

Original object is unaffected.

49. What is Function Composition?

Function composition means **combining multiple functions so the output of one becomes input of the next.**

Example

```
const double = x => x * 2;
const square = x => x * x;

const composed = x => square(double(x));

console.log(composed(3));
```

Output

JAVASCRIPT Copy

```
36
```

This concept is widely used in:

- functional programming
- Redux
- middleware pipelines

---

50. Practical Use Case of Functional Composition

Example pipeline:

JAVASCRIPT Copy

```
const getName = user => user.name;
const upperCase = str => str.toUpperCase();
const firstFive = str => str.slice(0,5);

const process = pipe(
  getName,
  upperCase,
```

```
const user = {
  name: "Geeky Frontend"
};

console.log(process(user));
```

Output

JAVASCRIPT Copy

```
GEEKY
```

This approach makes code:

- reusable
- modular
- easier to test

QUICK CHECK — no marks, just for you.

1. The lesson says a closure lets an inner function remember variables from the scope where it was created. What does this snippet print?

JAVASCRIPT Copy

```
function outer(x) {
  let count = x;
  function inner() {
    count++;
    console.log(count);
  }
  return inner;
}

const fn = outer(2);
fn();
```

```
fn2();
```

- 3
  4
  3
- 3
  3
  3
- 3
  4
  5
- 2
  3
  2

Check answer

2. According to the lesson, what structurally distinguishes event delegation from attaching a handler to each child element?

- A listener is attached to every child, but bubbling is disabled so only one of them fires
- The parent listener replaces bubbling entirely, so child elements can no longer receive click events
- A single listener is attached to a parent element, and the handler inspects event.target to decide what was clicked
- The listener is attached in the capture phase, which prevents the event from reaching the children

Check answer

3. Hoisting: logging x before 'var x = 5' prints undefined, but logging a before 'let a = 5' throws a ReferenceError. Why do the two differ?

- let declarations are not hoisted at all, so the name a does not exist anywhere before the declaration line
- let is hoisted too, but it sits in the Temporal Dead Zone and cannot be read until its declaration is evaluated
- var is hoisted and initialised to undefined, while let is hoisted and initialised to null, and reading null throws
- let is only hoisted inside functions, so at the top level the declaration stays exactly where it is written

Check answer

‹ Previous
JS and React Patterns and Solid Principles

Next ›
Data Types & Coercion

LEARN MORE

ARTICLES & DOCS

- JS interview questions (repo)
  github.com
- The Modern JavaScript Tutorial
  javascript.info

VIDEOS



Top 100 JavaScript interview questions
YouTube

50 JS interview questions in 1 hour

JS JavaScript Interview Kit



Concepts you must know

YouTube

Hand-picked references for this lesson.